Министерство науки и высшего образования РоссийскойФедерации
Федеральное государственное бюджетноеобразовательное
учреждение высшего образования
«Алтайский государственный технический университет им. И. И.Ползунова»
Университетский технологический колледж

Отчет защищен с оценкой

Преподаватель И.*Ю. Бутов*

«_____» 2025 г

Курсовая работа

по дисциплине «Проектирование и разработка веб-приложений»
по теме: «Сайт для торговой площадки игр»

Студент группы 1ИСП - 21 *Тройнов А. В.*

Преподаватель *И.Ю. Бутов*

БАРНАУЛ 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3

1. Обзор предметной области.....4

1. 1 Описание предметной области.....4

1. 2 Обзор существующих решений.....4

1. 2. 1 FatSecrut как существующее решение.....4

2. Проектирование.....5

2.1 Функциональные требования.....5

2.2 Диаграмма прецедентов.....5

2.3 Концептуальная модель.....5

2.4 Логическая модель и ее описание.....5

2.5 Используемые технологии.....5

3. Реализация.....6

3.1 Диаграмма классов.....6

3.2 Интерфейс разрабатываемого решения.....6

3.3 Тестирование.....6

ЗАКЛЮЧЕНИЕ.....7

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....8

Приложение А Руководство пользователя.....10

Приложение Б руководство программиста.....11

Приложение В Исходный код.....12

ВВЕДЕНИЕ

В условиях стремительного развития цифровых технологий и интернета индустрия компьютерных и видеоигр занимает одно из ведущих мест на рынке цифровых продуктов. Современные пользователи все чаще приобретают игры, внутриигровые предметы и дополнительные материалы через онлайн-платформы, что делает разработку специализированных торговых площадок особенно актуальной. Магазин-сайт для продажи игр и сопутствующего контента является удобным инструментом как для продавцов, так и для покупателей, обеспечивая быстрый доступ к товарам, безопасные платежи и комфортное взаимодействие с пользователем.

Актуальность данной курсовой работы обусловлена растущим спросом на онлайн-магазины игрового контента, а также необходимостью создания функциональных, удобных и визуально привлекательных веб-ресурсов. Грамотно разработанный сайт торговой площадки игр должен учитывать требования пользователей, обеспечивать простоту навигации, надежность хранения данных и возможность масштабирования.

Целью курсовой работы является разработка магазина-сайта для торговой площадки игр, который будет включать основной функционал онлайн-продаж и соответствовать современным требованиям веб-разработки.

Для достижения поставленной цели в работе необходимо решить следующие задачи:

- проанализировать существующие торговые площадки игр и их функциональные особенности;
- определить требования к структуре и дизайну магазина-сайта;
- разработать структуру веб-сайта и его основные страницы;
- реализовать базовый функционал интернет-магазина;
- оценить удобство и практическую значимость разработанного решения.

Объектом исследования является процесс онлайн-продажи игрового контента.

Предметом исследования является разработка и функционирование магазина-сайта для торговой площадки игр.

Практическая значимость данной курсовой работы заключается в возможности использования разработанного сайта в качестве основы для реального онлайн-магазина или учебного проекта в области веб-разработки.

1 Обзор предметной области

1. 1 Описание предметной области

Современный рынок видеоигр стремительно развивается: количество игр, разработчиков и игроков увеличивается с каждым годом. В результате растёт спрос на удобные онлайн-площадки, позволяющие покупать, продавать и распространять цифровые игровые продукты. Такие площадки обеспечивают удобный доступ к играм, автоматизацию процессов покупки и скачивания, а также позволяют пользователям взаимодействовать друг с другом, оставляя отзывы и рейтинги.

Торговая площадка игр представляет собой веб-ресурс, обеспечивающий пользователей возможностью просматривать каталог игр, приобретать цифровые товары, хранить свои покупки в личном кабинете и общаться через отзывную систему. Основными участниками подобной системы являются:

- Зарегистрированный пользователь — пользователи, приобретающие игры. Они могут просматривать каталог, фильтровать игры по жанрам, изучать карточки товаров, добавлять товары в корзину и совершать покупки.
- Не зарегистрированный пользователь — пользователь, который имеет доступ только к просмотру новостного канала и регистрации, для открытия большего спектра функций.

Внутри такого сервиса реализуется множество взаимосвязанных процессов:

- формирование каталога;
- отображение карточек игр с подробной информацией и ценой;
- покупка игр и оформление заказов через корзину;
- хранение истории приобретённых продуктов в личном кабинете;

Одной из ключевых задач является обеспечение стабильной и безопасной работы сайта, поскольку торговая площадка оперирует пользовательскими данными, транзакциями и цифровыми товарами. Кроме того, важно обеспечить удобный интерфейс, понятную навигацию и высокую скорость работы сервиса.

1. 2 Обзор существующих решений

На рынке существует большое количество площадок, предоставляющих пользователям доступ к цифровым играм. Они имеют разные модели распространения, функциональность и пользовательский опыт. Анализ существующих решений позволяет определить требования к собственной реализации, а также выявить подходы, которые оказались успешными или, наоборот, неэффективными.

К основным критериям анализа аналогов относятся:

- удобство интерфейса и логика пользовательских сценариев;
- структура каталога и навигации;
- способы поиска и фильтрации игр;
- система пользовательских отзывов и рейтингов;
- надёжность платформы и уровень защиты данных;
- техническая архитектура (клиент, сервер, API);
- особенности взаимодействия с разработчиками и пользователями.

Рассмотрим несколько наиболее известных торговых площадок цифровых игр.

1. 2. 1 Steam Marketplace как существующее решение

Steam Marketplace является одной из крупнейших и наиболее популярных платформ для распространения цифровых игр. Сервис предоставляет широкий функционал: покупку игр, маркетплейс внутриигровых предметов, социальные возможности, систему достижений, облачное хранение сохранений и многое другое, но главным примером, который отробразится на сайте — будет возможность покупать игр или игровые предметы, а также смотреть новости связанные с играми.

Основные преимущества Steam:

- огромный каталог игр разных жанров;
- удобная система рекомендаций и персонализации;
- развитая инфраструктура для разработчиков (Steamworks);
- развитая социальная экосистема: профили, друзья, чат, мастерская;

- регулярные скидки и распродажи.

Однако у платформы есть и недостатки:

- высокая комиссия для разработчиков;
- громоздкий интерфейс для некоторых категорий пользователей;
- ограниченные возможности настройки для независимых продавцов.

2 ПРОЕКТИРОВАНИЕ

2.1 Функциональные требования

Функциональные требования определяют набор возможностей, которые должен предоставлять разрабатываемый магазин-сайт для торговой площадки игр. Требования формируются на основе анализа предметной области и существующих аналогов. Основные функциональные требования к системе включают:

- регистрацию и авторизацию пользователей;
- просмотр каталога игр с возможностью;
- просмотр подробной информации об игре (название, описание, цена);
- добавление игр в корзину;
- оформление заказа и покупка игр;
- хранение истории покупок в личном кабинете пользователя;

Дополнительно система должна обеспечивать корректную обработку пользовательских данных и устойчивую работу при одновременном доступе нескольких пользователей.

2.2 Диаграмма прецедентов

Диаграмма прецедентов используется для наглядного представления взаимодействия пользователей с системой и описывает основные сценарии использования магазина-сайта. В разрабатываемой системе выделяются следующие акторы – гость и зарегистрированный пользователь.

Гость системы имеет возможность просматривать каталог игр и карточки товаров, а также регистрироваться в системе.

Зарегистрированный пользователь может выполнять вход в систему, добавлять игры в корзину, оформлять заказы, просматривать историю покупок.

Диаграмма прецедентов позволяет определить границы системы и основные функции, которые будут реализованы на этапе программной реализации.

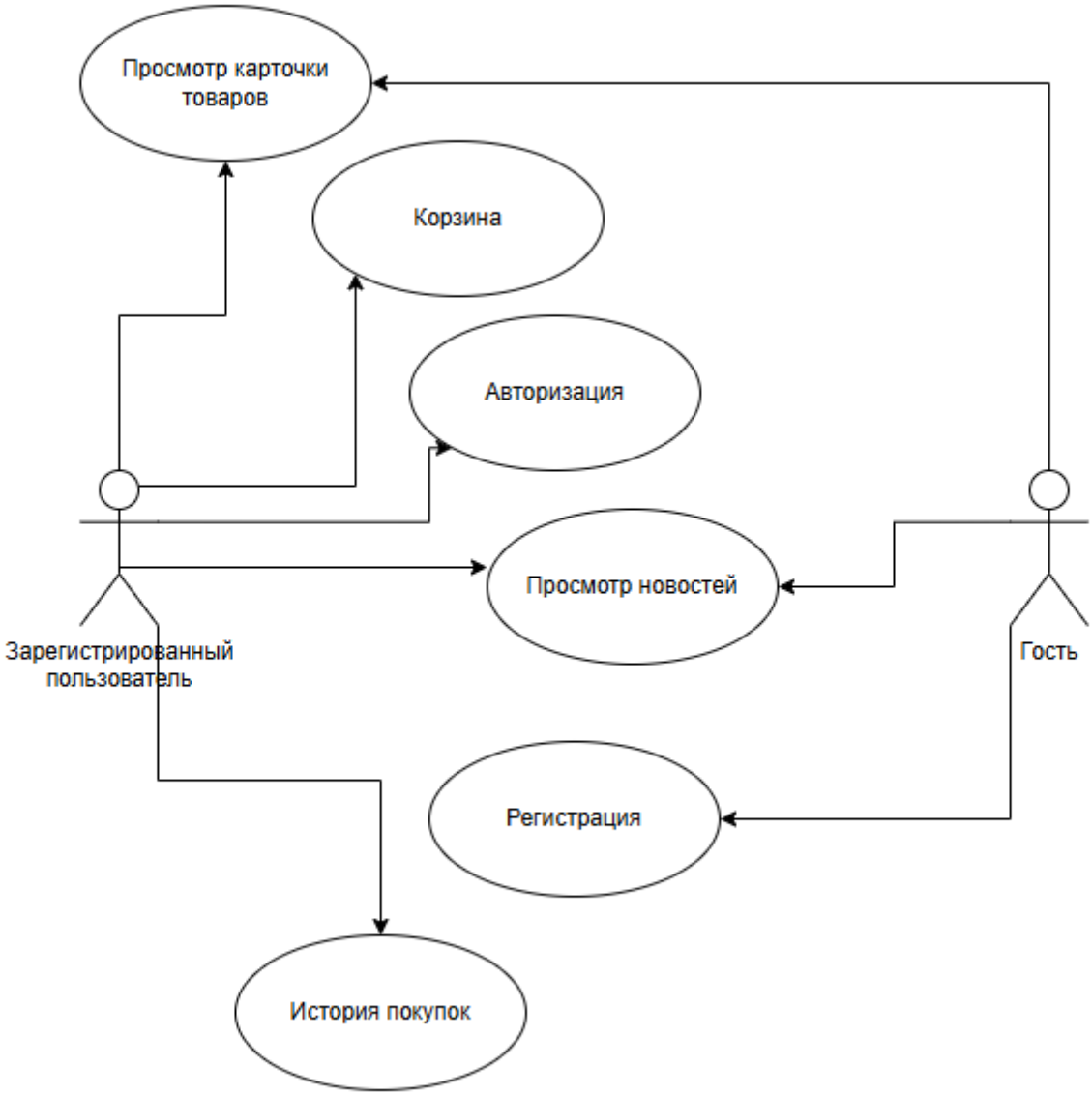

Рисунок 1 — Диаграмма прецедентов

2.3 Концептуальная модель

Концептуальная модель описывает основные сущности предметной области и связи между ними без привязки к конкретной реализации или СУБД.

Основными сущностями разрабатываемой торговой площадки являются:

- Пользователь;
- Игра;
- Игровой предмет;

- Корзина;
- Заказ;
- История.

Пользователь может оформлять несколько заказов, каждый заказ может содержать одну или несколько игр. Корзина используется для временного хранения выбранных игр и игровых предметов до момента оформления заказа.

Концептуальная модель позволяет на раннем этапе определить структуру данных и логические связи между объектами системы.

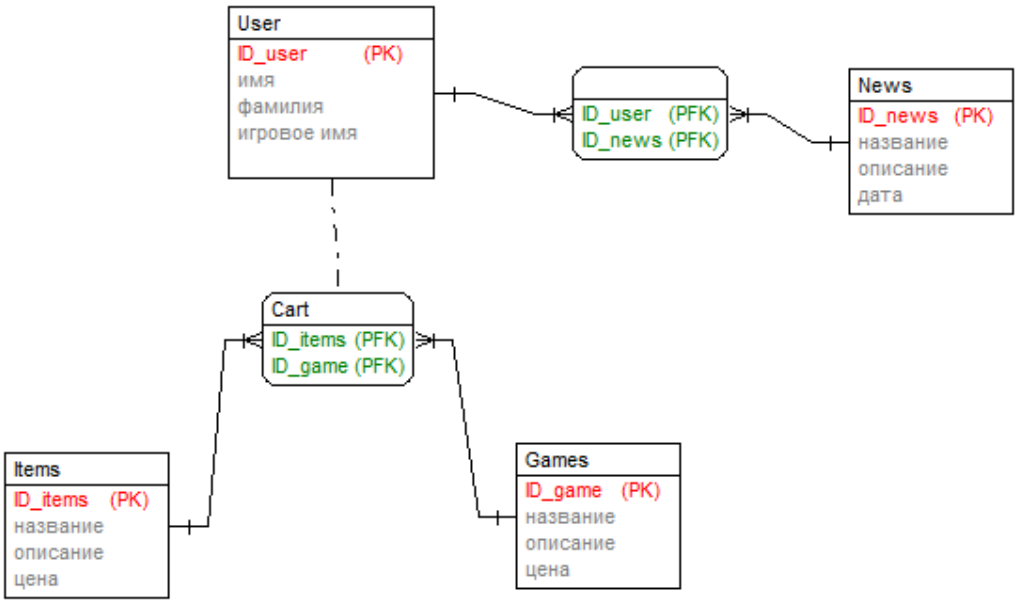

Рисунок 2 — Концептуальная модель

2.4 Логическая модель и ее описание

Логическая модель является уточнением концептуальной модели и отражает структуру данных с учётом требований к хранению информации. Она описывает таблицы базы данных, их атрибуты и связи между ними.

В рамках логической модели предполагается использование следующих основных таблиц:

Users — хранение данных пользователей (id, логин, имя, фамилия);

Games — информация об играх (id, название, описание, цена);

Items — информация об игровых товарах (id, название, описание, цена);

News — новости (id, название, описание, дата);

Cart — список корзины;

Связи между таблицами реализуются с использованием первичных и внешних ключей, что обеспечивает целостность и согласованность данных.

2.5 Используемые технологии

Выбор используемых технологий формирует технический фундамент разрабатываемой системы и определяет возможности её дальнейшего развития и масштабирования. В качестве серверной части применяется Django — современный высокоуровневый веб-фреймворк, основанный на языке программирования Python. Он предоставляет удобные инструменты для работы с базами данных, включает встроенную административную панель, а также обеспечивает обработку пользовательских запросов и взаимодействие с клиентской частью приложения. Благодаря продуманной архитектуре Django хорошо подходит для создания приложений с чётко организованной логикой и структурированными данными.

Для реализации клиентской части используется библиотека React, предназначенная для разработки динамических пользовательских интерфейсов. React позволяет создавать отзывчивые веб-приложения, эффективно управлять состоянием интерфейса и обновлять отображаемые данные без необходимости полной перезагрузки страницы.

Разделение системы на серверную и клиентскую части обеспечивает гибкость архитектуры, упрощает процесс масштабирования и позволяет вносить изменения в отдельные компоненты системы без затрагивания её основной логики.

3. РЕАЛИЗАЦИЯ

3.1 Диаграмма классов

Диаграмма классов отражает объектно-ориентированную структуру разрабатываемой системы и демонстрирует основные классы, их атрибуты, методы и взаимосвязи между ними. Данная диаграмма разрабатывалась с учётом концептуальной и логической моделей, описанных в предыдущей главе.

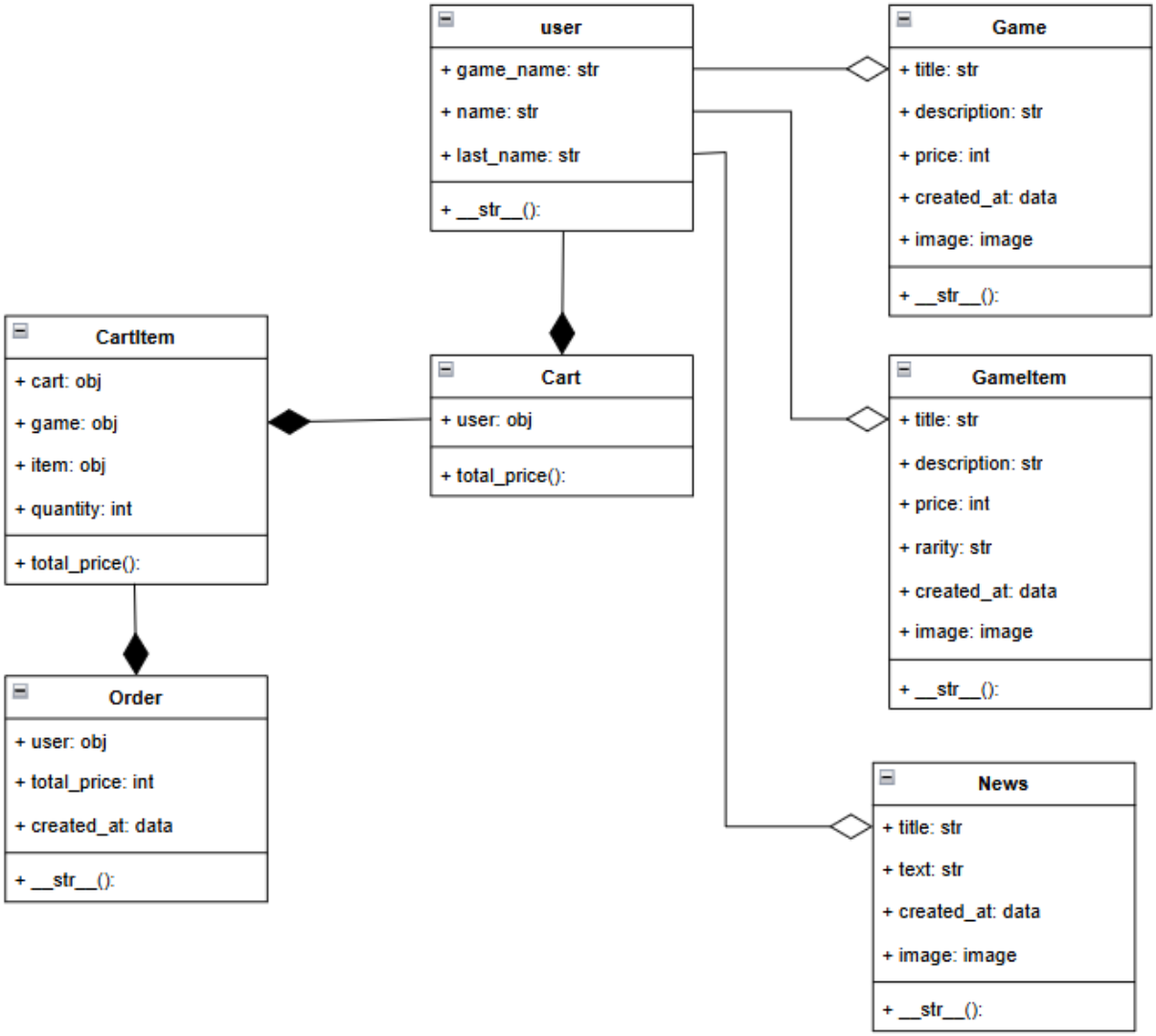

Рисунок 3 — диаграмма классов

В системе выделены следующие основные классы:

User — представляет пользователя системы и содержит данные, необходимые для идентификации и работы с личным кабинетом:

- game_name:str
- name:str
- last_name:str

Game — описывает игру, доступную для покупки, включая название, описание и стоимость:

- title:str
- description:str
- price:int
- created_at:date
- image: img

GameItem — класс, предназначенный для представления игровых предметов;

- title:str
- description:str
- price:int
- rarity:str
- created_at:date
- image: img

Cart — используется для хранения выбранных пользователем товаров перед оформлением заказа;

- user:obj

CartItem — используется для хранения выбранных пользователем товаров перед оформлением заказа;

- cart:obj
- game:obj
- item:obj
- quanity:int

Order — заказ, который имеет содержимое корзины;

- user:obj
- total_price:int
- last_name:str

News — отвечает за публикацию и отображение новостей, связанных с играми и площадкой.

- game_name:str
- name:str
- created_at:date

Класс User связан с классом Cart отношением 1 к N, так как один пользователь может оформить несколько заказов. Класс Cart связан с классами Game и Item, поскольку заказ может включать несколько игровых товаров. Диаграмма классов позволяет наглядно представить структуру приложения и служит основой для реализации серверной логики.

3.2 Интерфейс разрабатываемого решения

Интерфейс разрабатываемого магазина-сайта ориентирован на удобство использования и интуитивно понятную навигацию. При проектировании пользовательского интерфейса учитывались типовые сценарии взаимодействия пользователя с торговой площадкой. Главная страница сайта содержит:

- блок магазина игр;
- блок магазина игровых предметов;
- блок новостей.

Для зарегистрированных пользователей доступен личный кабинет, в котором отображается история покупок и текущая корзина. Карточка товара содержит подробное описание, цену и кнопку добавления в корзину.

Клиентская часть реализована с использованием библиотеки React, что позволяет динамически обновлять содержимое страниц без полной перезагрузки. Это повышает отзывчивость интерфейса и улучшает пользовательский опыт. Взаимодействие с серверной частью осуществляется посредством HTTP-запросов к API, реализованному на стороне Django.

Пример реализации клиентской части (рисунки 5 — 11).

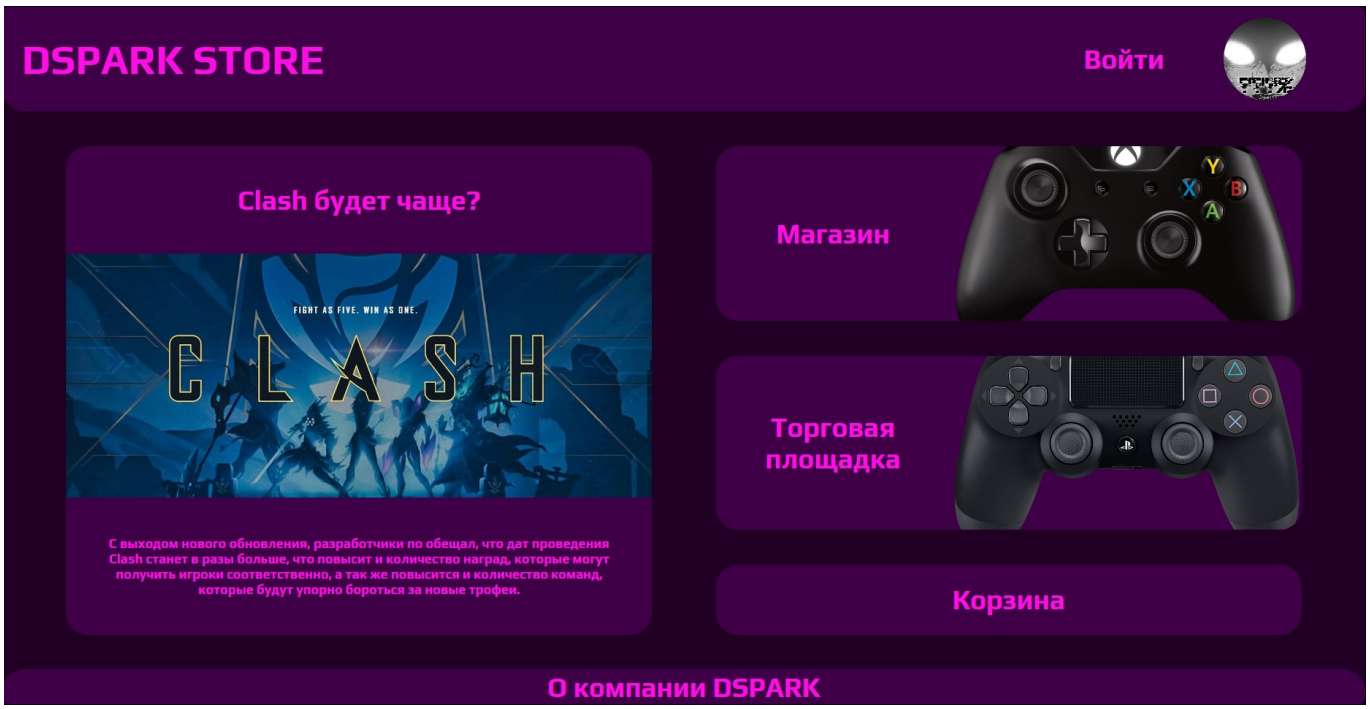

Рисунок 4 — главная страница

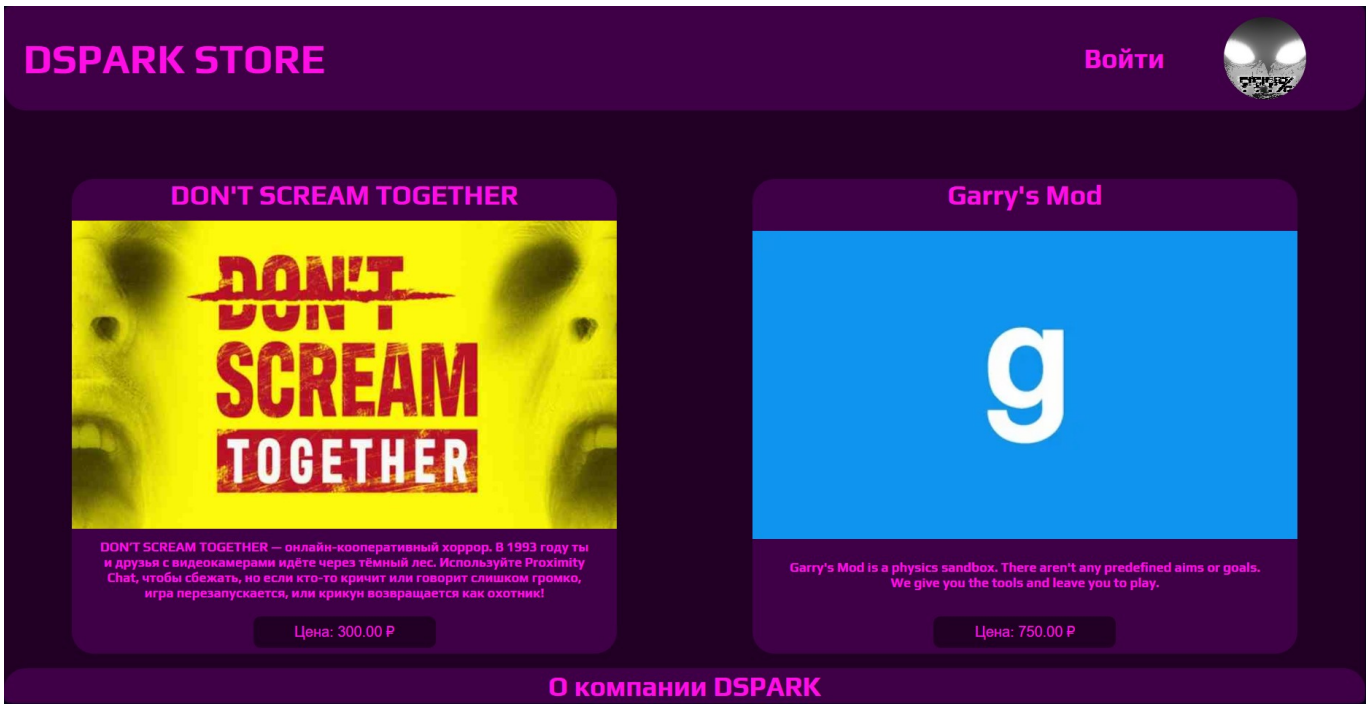

Рисунок 5 — страница магазина

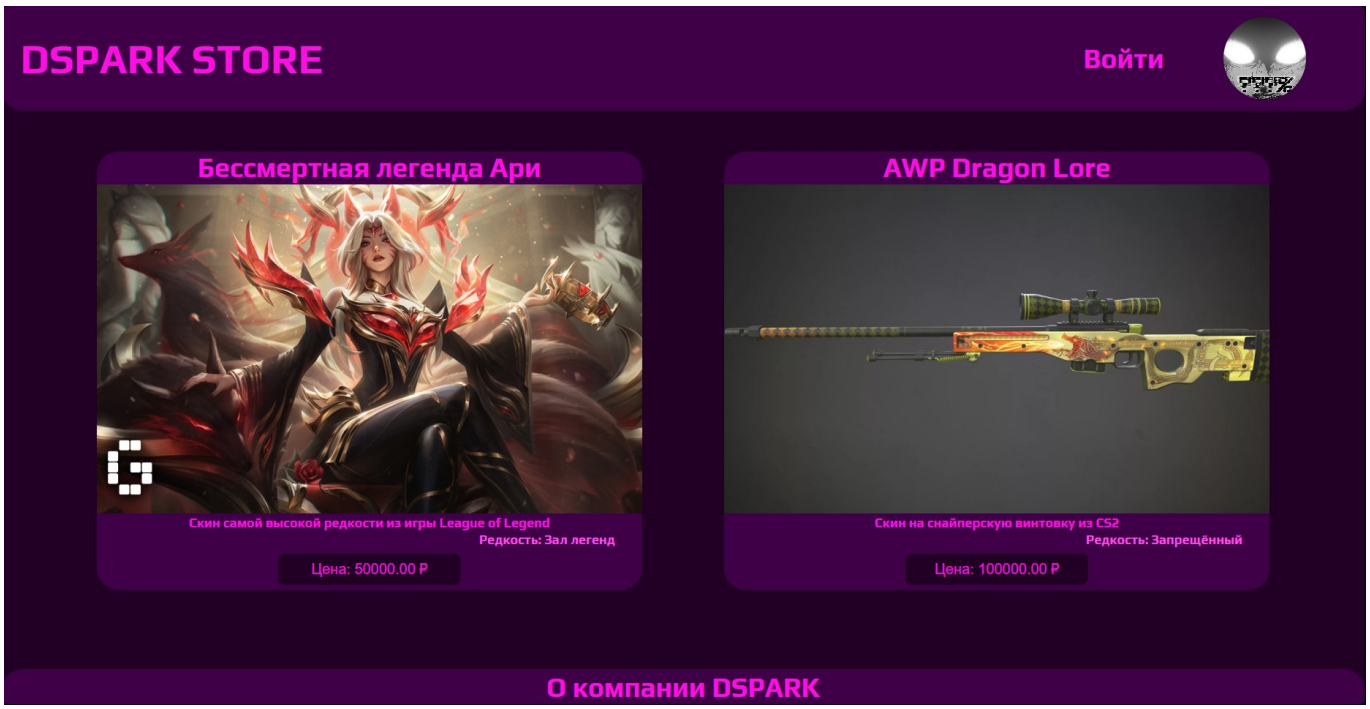

Рисунок 6 — страница торговой площадки

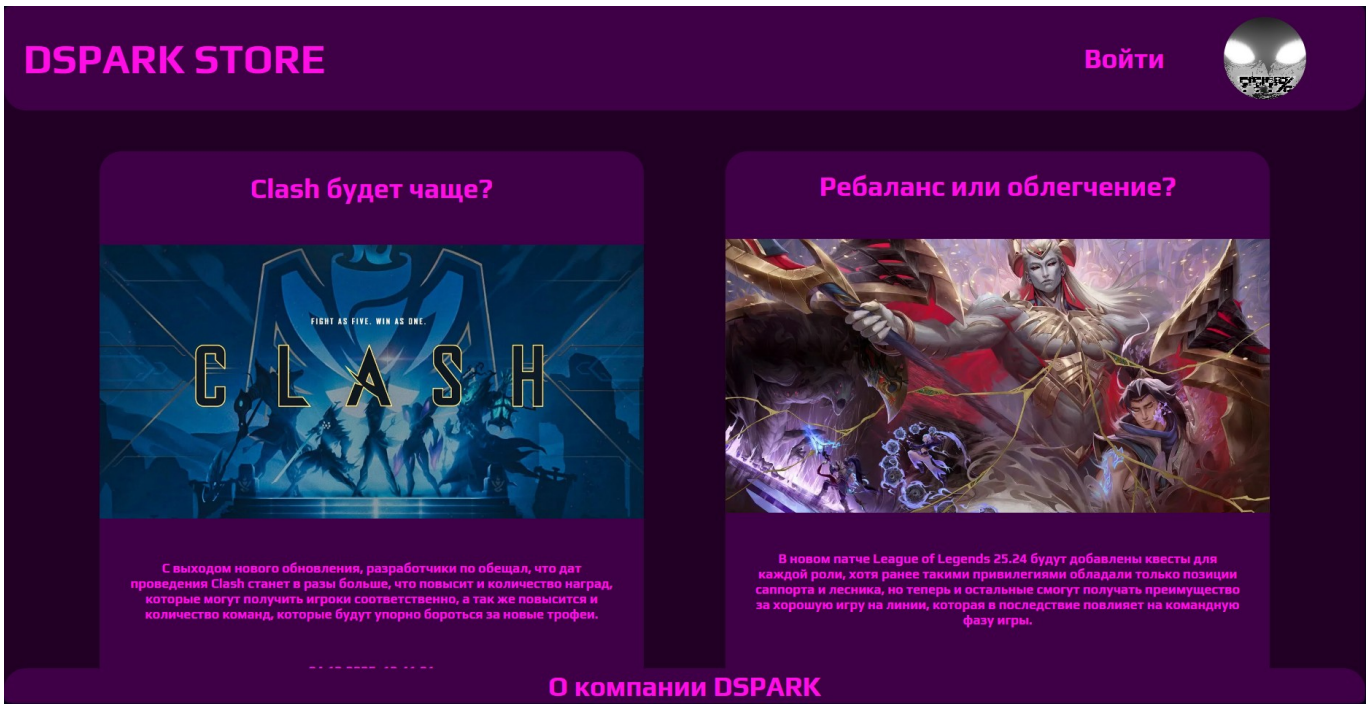

Рисунок 7 — новостная страница

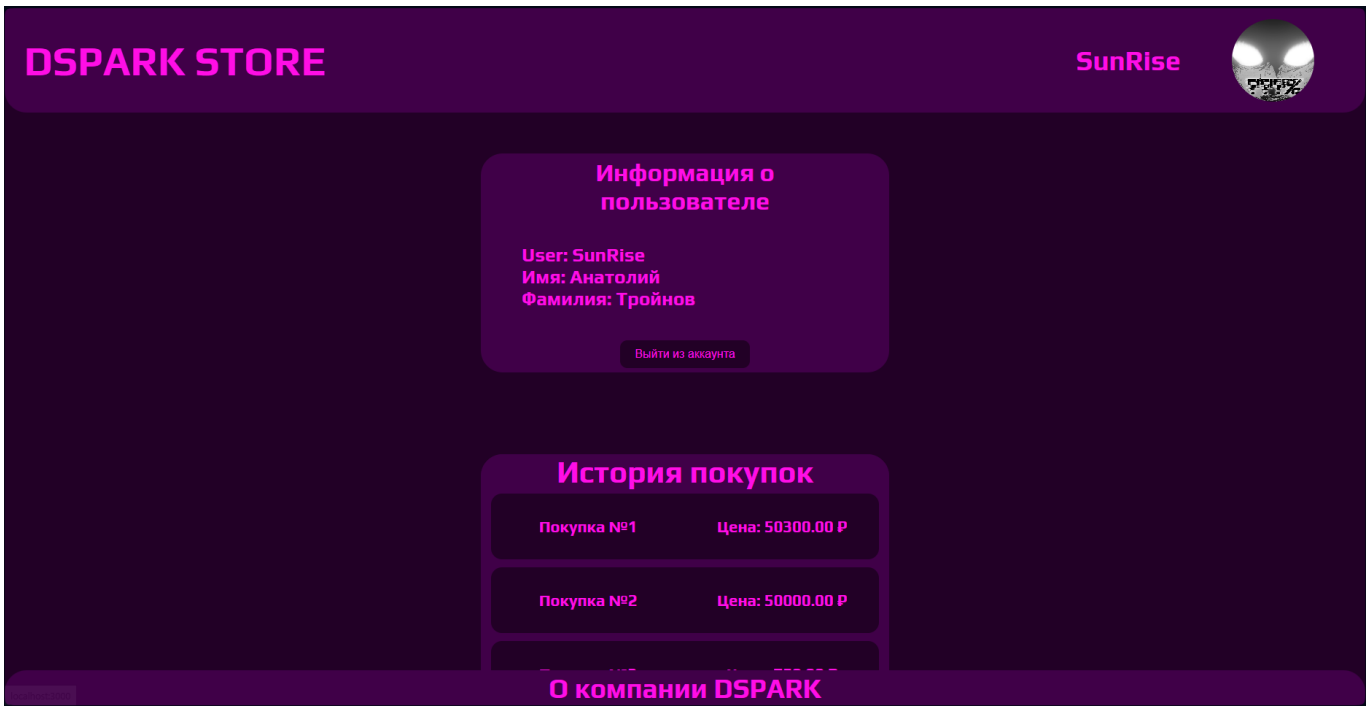

Рисунок 8 — профиль

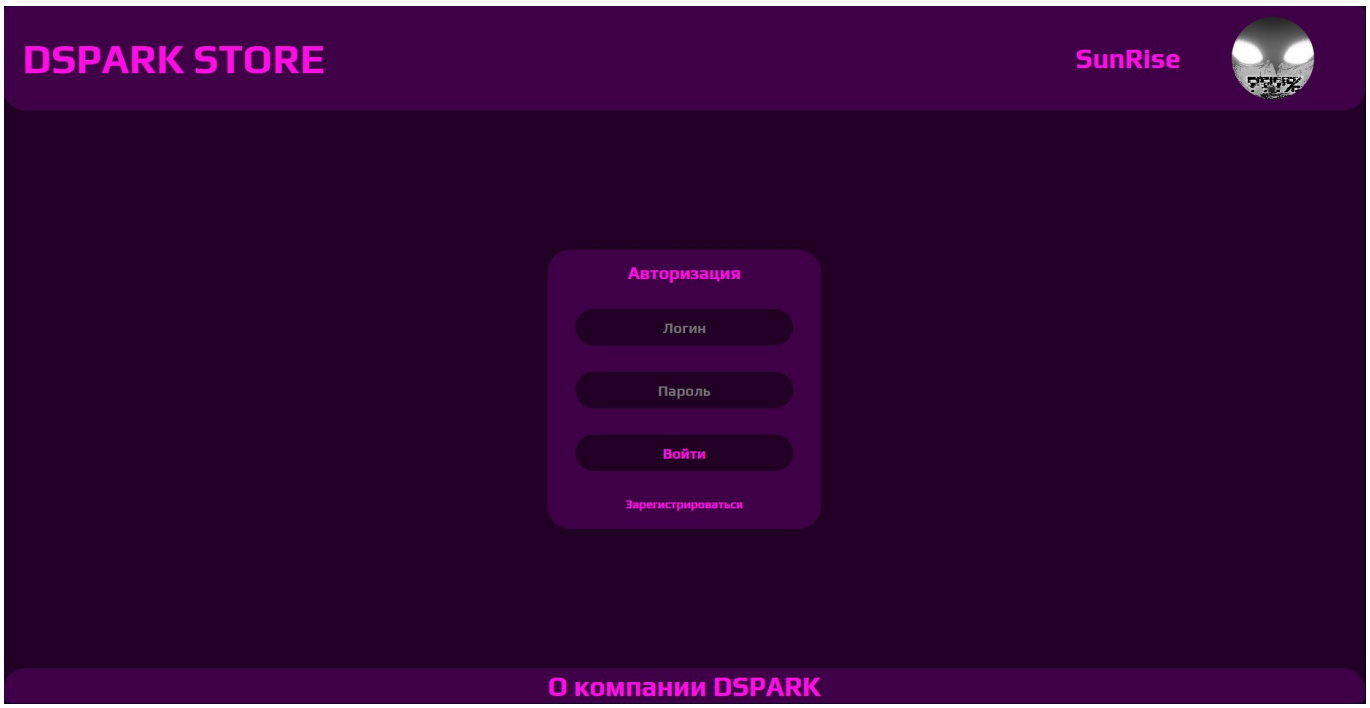

Рисунок 9 — авторизация

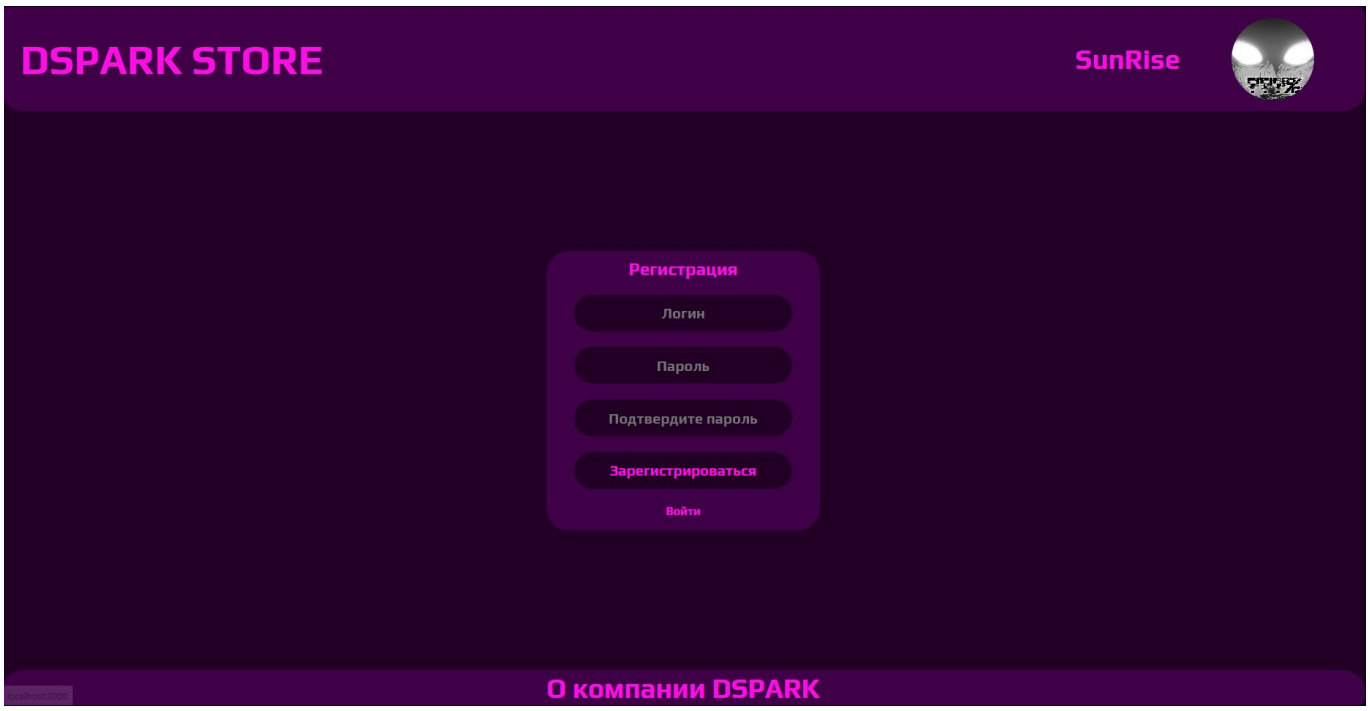

Рисунок 10 — регистрация

3.3 Тестирование

Тестирование является важным этапом разработки и направлено на проверку корректности работы реализованного функционала. В ходе тестирования проверялись основные пользовательские сценарии работы с сайтом. Были выполнены следующие виды тестирования:

- проверка регистрации и авторизации пользователей;
- тестирование добавления товаров в корзину;
- проверка оформления заказа;
- проверка корректного отображения каталога игр и карточек товаров;
- тестирование работы личного кабинета и истории покупок.

По результатам тестирования было установлено, что реализованный магазин-сайт корректно выполняет все заявленные функции, устойчиво работает при стандартной пользовательской нагрузке и соответствует функциональным требованиям, сформулированным на этапе проектирования.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана модель магазина-сайта для торговой площадки игр, ориентированная на продажу цифровых игровых продуктов и внутриигровых предметов. Работа охватывает основные этапы создания веб-приложения — от анализа предметной области до проектирования и реализации функционала системы.

В первой главе был проведён обзор предметной области, рассмотрены особенности рынка цифровых игр и принципы функционирования торговых площадок. Также был выполнен анализ существующих решений на примере Steam Marketplace, что позволило выявить ключевые требования к разрабатываемому сайту и определить необходимый набор функций.

Во второй главе были сформированы функциональные требования к системе, разработаны диаграммы прецедентов, концептуальная и логическая модели данных. Кроме того, был обоснован выбор используемых технологий, обеспечивающих надёжность, гибкость и возможность дальнейшего развития проекта.

Третья глава была посвящена реализации разработанного решения. Были описаны структура классов системы, пользовательский интерфейс и проведено тестирование основных функций. Результаты тестирования показали, что разработанный магазин-сайт корректно реализует заявленный функционал и соответствует поставленным требованиям.

Таким образом, цель курсовой работы была достигнута, а все поставленные задачи — выполнены. Разработанный магазин-сайт может быть использован в качестве основы для дальнейшего развития полноценной торговой площадки игр, а также как учебный проект для изучения принципов веб-разработки и объектно-ориентированного программирования.

Практическая значимость работы заключается в возможности расширения функционала системы, внедрения дополнительных сервисов и адаптации проекта под реальные условия эксплуатации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Хоган Б. HTML5 и CSS3. Веб-разработка по стандартам нового поколения. / Б. Хоган. – 2-е издание. – Санкт-Петербург : Питер, 2019. – 320 с.
2. Хоган Б. HTML5 и CSS3. Веб-разработка по стандартам нового поколения. / Б. Хоган. – 2-е издание. – Санкт-Петербург : Питер, 2019. – 320 с.
3. Воройский, Ф. С. Информатика. Энциклопедический систематизированный словарь-справочник / Ф. С. Воройский – Москва : Физматлит, 2020. – 945 с.
4. UTEX : сайт : Что такое веб-сайт: виды, цели, особенности сайтов – 2020. – URL : https://utex.ru/chto-takoe-veb-sayt/ (дата обращения: 18.12.2024). – Режим доступа: свободный.
5. Голубев, Е. Домен и Хостинг / Е. Голубев. – Ridero, 2020. – 26 с. – Текст : непосредственный.
6. Гоше, Х. Д. HTML5. Для профессионалов. / Х. Д. Гоше. – Санкт-Петербург : Питер, 2021. – 496 с. : ил.
7. Дакетт, Д. HTML и CSS. Разработка и дизайн веб-сайтов / Д. Дакетт ; перевод с английского М. А. Райтмана. – Москва : Эксмо, 2021. – 480 с. : ил. + 1 CD. – (Мировой компьютерный бестселлер).
8. Что такое виртуальный хостинг: отличия от виртуального сервера и где лучше разместить сайт / М. Талипов – Текст : электронный // VK Cloud. – 2020. – URL : https://cloud.vk.com/blog/virtualnyy-khosting-otlichiya-ot-virtualnogo-servera (дата обращения: 18.12.2024). – Режим доступа: свободный.
9. Дронов В. А. HTML 5, CSS 3 и Web 2.0. Разработка современных Web-сайтов / В. А. Дронов. – Санкт-Петербург : БХВ-Петербург, 2019. – 416 с. : ил. – (Профессиональное программирование).
10. Квинт И. HTML, XHTML и CSS на 100 % / И. Квинт. – Санкт-Петербург : Питер, 2020. – 384 с.
11. Клименко, Р. Веб-мастеринг на 100% / Р. Клименко. – Санкт-Петербург : Питер, 2021. – 512 с.

12. Лоусон Б. Изучаем HTML5. Библиотека специалиста. / Б. Лоусон, Р. Шарп. – 2-е издание. – Санкт-Петербург : Питер, 2022. – 304 с. : ил.
13. Мак-Дональд М. HTML5. Недостающее руководство / М. Мак-Дональд. – Санкт-Петербург : БХВ-Петербург, 2022. – 480 с.
14. Макфарланд Д. Большая книга CSS3 / Д. Макфарланд. – 3-е издание. – Санкт-Петербург : Питер, 2019. – 608 с.
15. Никсон, Р. Создаем динамические веб-сайты PHP, MySQL, JavaScript, CSS и HTML5 / Р. Никсон. – 4-е издание. – Санкт-Петербург : Питер, 2019. – 768 с.
16. Роббинс, Д. HTML5, CSS3 и JavaScript. Исчерпывающее руководство / Д. Роббинс ; перевод с английского М. А. Райтмана. – 4-е издание. – Москва : Эксмо, 2020. – 528 с.
17. Серго, А. Доменные имена / А. Серго. – Москва : Бестселлер, 2020. – 368 с.
18. Татро К. Создаем динамические веб-сайты на PHP. / К. Татро, П. Макинтайр. – 4-е издание. – Санкт-Петербург : Питер, 2021. – 544 с.
19. КОД. Журнал Яндекс Практикума : сайт : Делаем неубиваемый сайт : статика и динамика – URL : https://thecode.media/static/ (дата обращения: 16.12.2024). – Режим доступа: свободный.
20. Фримен, Э. Изучаем HTML, XHTML и CSS / Э. Фримен, Э. Фримен. – Санкт-Петербург : Питер, 2022. – 656 с.

ПРИЛОЖЕНИЕ А РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Для попадания на главный экран после всех последующих действий, нажмите на название сайта, находящееся в шапке сайта, с левой стороны.

Авторизация и Регистрация:

1. Для того, что бы авторизоваться, нужно нажать на кнопку войти, в шапке сайт, слева от иконки;
2. После попадания на страницу с формой авторизации нужно ввести свои данные, после чего нажать войти;
3. В случае, если вы не зарегистрированы, под кнопкой войти есть гипер-ссылка с надписью «регистрация»;
4. После попадания на форму для регистрации, вы должны придумать свой логин, а также пароль;
5. После чего вам необходимо ввести их и обязательно подтвердить пароль;
6. После проделанных действий, вы можете вернуться на страницу авторизации при помощи гипер-ссылки с надписью «войти»;
7. После этого вам нужно заново произвести действия под номером 1 и 2 в данной инструкции, после этого вы сможете пользоваться всеми функциями сайта.

Просмотр профиля:

1. Для попадания в профиль нужно найти в шапке, с правой стороны иконку профиля и нажать по ней, после чего вы окажитесь на странице профиля;
2. В верхней части экрана вы можете видеть вашу личную информацию, а ниже будет расположена история ваших покупок.

Просмотр истории покупок:

1. Для того что бы посмотреть историю покупок, в должны выполнить действия для просмотра профиля, описанные выше;

2. После попадания на страницу профиля, вам нужно держа экран на темном фоне сайта, прокрути колёсико мыши вниз, после чего вам в полной мере откроется блок с вашей историей покупок;
3. Если покупок накопилось много наведите мышь на блок истории покупок, прокрутите колёсиков мыши вниз и вы сможете увидеть более старые покупки.

Просмотр корзины:

1. Для того, что бы перейти в корзину вам нужно найти в самом низу правой колонки длинную кнопку корзина и нажать по ней;
2. После чего вы окажетесь на странице корзины.
3. Для оформления покупки нажмите на кнопку купить;
4. В случае желания удалить не нужный или случайно выбранный товар, нажмите на кнопку удалить, рядом с товаром.

Просмотр новостей:

1. Нажать на новость в левой части экрана;
2. Оказавшись на странице новостей, листайте вниз и читайте описание новости находящееся под картинкой.
3. Для просмотра даты выложенной новости, обратите внимание на содержимое под текстом новости, там будет находиться дата.

Просмотр внутри-игровых товаров:

1. Нажать на кнопку торговая площадка в правой-нижней части экрана;
2. Оказавшись на странице торговая площадка, листайте вниз и читайте описание игровых товаров находящееся под картинкой;
3. При желании купить предмет, нажмите на кнопку купить, под описанием и он добавится в корзину.

Просмотр игровых товаров:

1. Нажать на кнопку магазин в правой-верхней части экрана;
2. Оказавшись на странице магазина, листайте вниз и читайте описание игры находящееся под картинкой;

3. При желании купить игру, нажмите на кнопку купить, под описанием и игра добавится в корзину.

ПРИЛОЖЕНИЕ Б РУКОВОДСТВО ПРОГРАММИСТА

Структура проекта

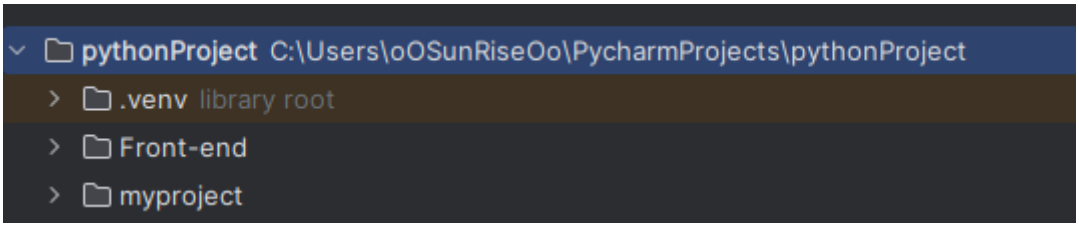

Рисунок 11 — основной каталог

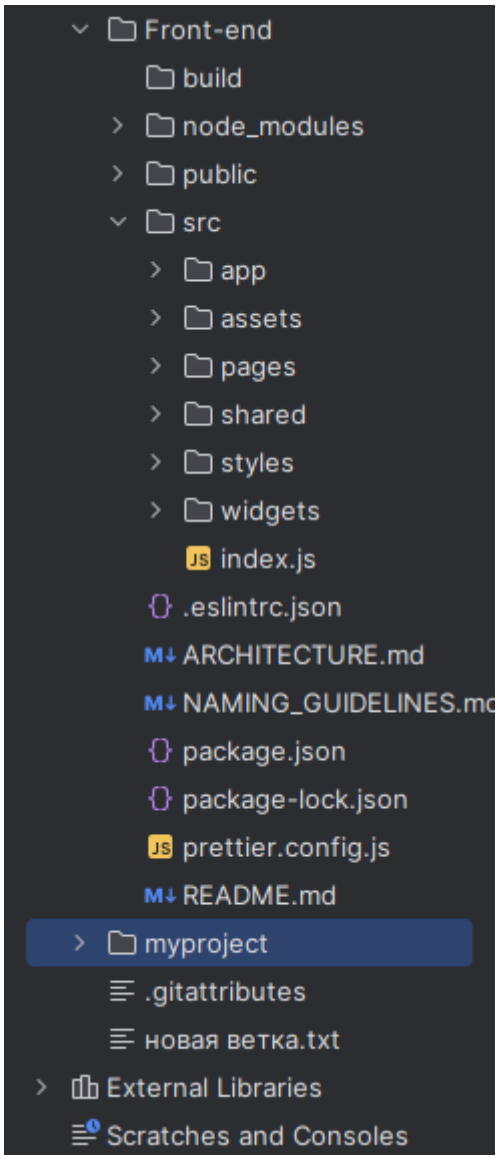

Рисунок 12 — строение каталога Front-end

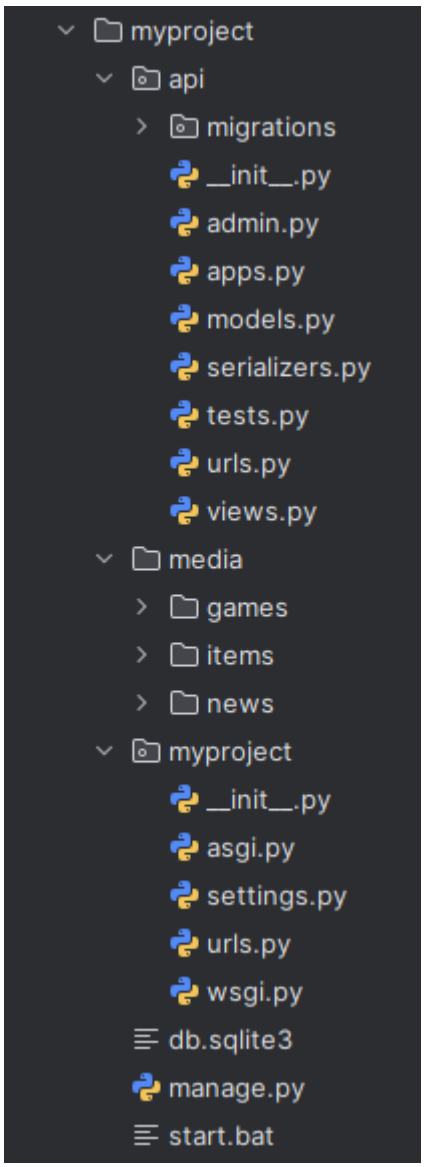

Рисунок 13 — строение каталога myproject(back-end).

Среда разработки и зависимости

Клиент (frontend):

- Node.js
- React
- react-router-dom

Сервер (backend):

- Python 3.10+
- Django 4.x
- Django REST Framework (DRF) — для построения API

ПРИЛОЖЕНИЕ В ИСХОДНЫЙ КОД

Листинг В.1 — файл index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>DSpark Store</title>

    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?
family=Bebas+Neue&family=Doto:wght@100..900&family=Jua&family=Kavoon&family=Lilita+One&family=Play:wght@
400;700&family=Shantell+Sans:ital,wght@0,300..800;1,300..800&display=swap" rel="stylesheet">
  </head>
  <body>
    <noscript>You need to enable JavaScript to run this app.</noscript>
    <div id="root"></div>
    <!--
      This HTML file is a template.
      If you open it directly in the browser, you will see an empty page.

      You can add webfonts, meta tags, or analytics to this file.
      The build step will place the bundled scripts into the <body> tag.

      To begin the development, run `npm start` or `yarn start`.
      To create a production bundle, use `npm run build` or `yarn build`.
    -->
  </body>
</html>
```

Листинг В.2 — файл index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { QueryClientProvider } from "@tanstack/react-query";
import { queryClient } from "./app/providers/queryClient.js";
import { ReactQueryDevtools } from "@tanstack/react-query-devtools";
```

```
import { BrowserRouter, Routes, Route} from "react-router-dom";
import './styles/index.css';
import reportWebVitals from './app/providers/reportWebVitals.js';
import Header from './widgets/header.js';
import Footer from './widgets/footer.js';
import MainPage from './pages/MainPage.js';
import NewsPage from './pages/NewsPage.js';
import ShopPage from './pages/ShopPage.js';
import ItemPage from './pages/ItemPage.js';
import CompanyPage from './pages/CompanyPage.js';
import ProfilePage from './pages/ProfilePage.js';
import RegisterPage from './pages/RegisterPage.js';
import LoginPage from './pages/LoginPage.js';
import CartPage from './pages/CartPage.js';
import { Provider } from "react-redux";
import { store } from "./app/store/store.js";

function App() {
 return(
 <div id="app">
    <BrowserRouter>
      <Header />
      <Routes>
        <Route index element={<MainPage />} />
        <Route path="/register" element={<RegisterPage />} />
        <Route path="/login" element={<LoginPage />} />
        <Route path="/profile" element={<ProfilePage />} />
        <Route path="/cart" element={<CartPage />} />
        <Route path="/news" element={<NewsPage />} />
        <Route path="/shop" element={<ShopPage />} />
        <Route path="/item" element={<ItemPage />} />
        <Route path="/company" element={<CompanyPage />} />
      </Routes>
      <Footer />
    </BrowserRouter>
    <ReactQueryDevtools initialIsOpen={false} />
 </div>
 )
}
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <Provider store={store}>
    <QueryClientProvider client={queryClient}>
      <App />
    </QueryClientProvider>
  </Provider>
);

// If you want to start measuring performance in your app, pass a function
// to log results (for example: reportWebVitals(console.log))
// or send to an analytics endpoint. Learn more: https://bit.ly/CRA-vitals
reportWebVitals(console.log);
```

Листинг В.3 — файл CartPage.js

```
import React, { useEffect } from "react";
import { useAppDispatch, useAppSelector } from "../app/store/hooks/reduxHooks";

import {
  fetchCart,
  removeCartItem,
  buyCart,
} from "../app/store/thunks/cartThunks";

function CartPage() {
  const dispatch = useAppDispatch();
  const cart = useAppSelector((state) => state.cart);

  useEffect(() => {
    dispatch(fetchCart());
  }, [dispatch]);

  function handleRemove(id) {
    console.log("REMOVE CLICK", id);
    dispatch(removeCartItem(id));
  }

  function handleBuy() {
```

```
    dispatch(buyCart());
  }

  if (!cart) return <p>Загрузка...</p>;

  return (
    <div id="content">
      <h2>Корзина</h2>

      {(!cart.items || cart.items.length === 0) && (
        <p>Корзина пуста</p>
      )}

      {cart.items?.map((item) => (
        <div key={item.id}>
          <p>
            {item.game?.title || item.item?.title} —{" "}
            {item.game?.price || item.item?.price} ₽
          </p>

          <button onClick={() => handleRemove(item.id)}>
            Удалить
          </button>
        </div>
      ))}

      <h3>Итого: {cart?.total ?? 0} ₽</h3>

      <button onClick={handleBuy}>
        Купить
      </button>
    </div>
  );
}

export default CartPage;
```

Листинг В.4 — файл CompanyPage.js

```
import React from 'react';
import '../styles/company.css';


function CompanyPage() {
  return(
  <div id="content">
      <div id="history_card">
        <p id="history_card_name">История компании</p>
        <p id="history_card_text">? ? ?</p>
      </div>
      <div id="user_accept_card">
        <p id="user_accept_card_name">Пользовательское соглашение</p>
        <p id="user_accept_card_text">? ? ?</p>
      </div>
  </div>
  )
}

export default CompanyPage;
```

Листинг В.5 — файл ItemPage.js

```
import React, { useEffect } from "react";
import "../styles/item.css";

import { useAppDispatch, useAppSelector } from "../app/store/hooks/reduxHooks";

import { fetchItems } from "../app/store/thunks/itemsThunks";
import { addItemToCart } from "../app/store/thunks/cartThunks";

function ItemPage() {
  const dispatch = useAppDispatch();

  const { items, loading, error } = useAppSelector(
    (state) => state.items
  );
```

```
const cartLoading = useAppSelector(
  (state) => state.cart.loading
);

// загрузка предметов
useEffect(() => {
  dispatch(fetchItems());
}, [dispatch]);

// добавление в корзину
const handleAddToCart = async (id) => {
  try {
    await dispatch(addItemToCart(id)).unwrap();

    // обновляем корзину после покупки
    dispatch(fetchItems());
  } catch (error) {
    alert("Ошибка: " + error);
  }
};

if (loading) return <p>Загрузка...</p>;
if (error) return <p>Ошибка загрузки предметов</p>;

return (
  <div id="content_items">

    {items.length === 0 && (
      <div id="item_card">
        <p id="item_card_name">Предметов пока нет</p>
        <p id="item_card_text">Зайдите позже</p>
      </div>
    )}

    {items.map((item) => (
      <div id="item_card" key={item.id}>
        <p id="item_card_title">{item.title}</p>

        <img
          src={`http://localhost:8000${item.image}`}
```

```
          alt="Нет изображения"
          id="item_card_img"
        />

        <p id="item_card_text">{item.description}</p>
        <p id="item_card_rarity">
          Редкость: {item.rarity}
        </p>

        <button
          id="buy_button"
          onClick={() => handleAddToCart(item.id)}
          disabled={cartLoading}
        >
          Купить: {item.price} ₽
        </button>
      </div>
    ))}

  </div>
 );
}

export default ItemPage;
```

Листинг В.6 — файл LoginPage.js

```
import React from 'react';
import '../styles/forms.css';
import { useForm } from 'react-hook-form';
import {Link} from 'react-router-dom';

function LoginPage() {
  const { register, handleSubmit, formState: { errors } } = useForm();

  async function SumbitForm(data) {
    const data_of_person = {
      username: data.login_name,
      password: data.password
    };

    console.log("Отправляем:", data_of_person);

    try {
      const response = await fetch("http://localhost:8000/api/login/", {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify(data_of_person)
      });

      const result = await response.json();
      console.log("Ответ Django:", result);

      if (response.ok) {
        localStorage.setItem("access", result.access);
        localStorage.setItem("refresh", result.refresh);

        alert("Вы успешно вошли!");
      } else {
        alert("Ошибка: Неверный логин или пароль");
      }
    } catch (error) {
      console.error("Ошибка запроса:", error);
```

```
            alert("Сервер недоступен");
        }
    }

    return (
        <div id="content_login">
            <form onSubmit={handleSubmit(SumbitForm)}>
                <p id="form_name">Авторизация</p>
                <input
                    id="form_input_place"
                    type="text"
                    placeholder="Логин"
                    {...register('login_name', { required: true })}
                />
                {errors.login_name && <p>Введите логин.</p>}

                <input
                    id="form_input_place"
                    type="password"
                    placeholder="Пароль"
                    {...register('password', { required: true })}
                />
                {errors.password && <p>Пароль обязателен.</p>}

                <input type="submit" value="Войти" id="accept_form_button"/>

                <p id="change_form_button"><Link to="/register">Зарегистрироваться</Link></p>
            </form>
        </div>
    );
}

export default LoginPage;
```

Листинг В.7 — файл MainPage.js

```
import React from 'react';
import '../styles/index.css';
import Shop_Image from '../assets/shop_logo.png';
import Item_Image from '../assets/item_logo.png';
import { Link } from "react-router-dom";
import LastNews from "../widgets/LastNews";

const MainPage = () => {
  return (
    <div id="content">

      <Link to="/news" id="first_blocks">
        <div id="news">
          <LastNews />
        </div>
      </Link>

      <Link to="/shop" id="second_blocks">
        <div id="game_shop" className="game_item_shop_card">
          <p className="name_card">Магазин</p>
          <img src={Shop_Image} alt="shop" id="second_blocks_image" />
        </div>
      </Link>

      <Link to="/item" id="second_blocks">
        <div id="item_store" className="game_item_shop_card">
          <p className="name_card">Торговая площадка</p>
          <img src={Item_Image} alt="items" id="second_blocks_image" />
        </div>
      </Link>

      <Link to="/cart" id="theerd_blocks">
        <div id="item_store" className="game_item_shop_card">
          <p className="name_card">Корзина</p>
        </div>
      </Link>

    </div>
```

```
  );
};

export default MainPage;
```

Листинг В.8 — файл NewsPage.js

```
import React from 'react';
import '../styles/news.css';
import { useNews } from '../shared/hooks/useNews';

function NewsPage() {

  const {
    data: news = [],
    isLoading,
    error
  } = useNews();

  if (isLoading) return <p>Загрузка...</p>;
  if (error) return <p>Ошибка загрузки новостей</p>;

  return (
    <div id="content_news">

      {news.length === 0 && (
        <div id="news_card">
          <p id="news_card_text">Новостей пока нет...</p>
        </div>
      )}

      {news.map(item => (
        <div id="news_card" key={item.id}>
          <p id="news_card_title">{item.title}</p>

          <img
            src={`http://localhost:8000${item.image}`}
            alt="Нет изображения"
            id="news_card_img"
          />
```

```
        <p id="news_card_text">{item.text}</p>

        <p id="news_card_text">
          {new Date(item.created_at).toLocaleString()}
        </p>
      </div>
    ))}

  </div>
 );
}

export default NewsPage;
```

Листинг В.9 — файл ProfilePage.js

```
import React, { useEffect } from "react";
import "../styles/profile.css";
import { useNavigate } from "react-router-dom";

import { useAppDispatch, useAppSelector } from "../app/store/hooks/reduxHooks";

import { fetchUser } from "../app/store/thunks/authThunks";
import { fetchOrders } from "../app/store/thunks/ordersThunks";
import { logout } from "../app/store/slices/authSlice";

function ProfilePage() {
 const navigate = useNavigate();
 const dispatch = useAppDispatch();

 // данные пользователя
 const {
  user,
  loading: profileLoading,
  error: profileError,
 } = useAppSelector((state) => state.auth);

 // заказы
 const {
```

```
  orders,
  loading: ordersLoading,
  error: ordersError,
} = useAppSelector((state) => state.orders);

// загрузка данных
useEffect(() => {
  dispatch(fetchUser());
  dispatch(fetchOrders());
}, [dispatch]);

// logout
function handleLogout() {
  dispatch(logout());
  navigate("/login");
}

// loading
if (profileLoading || ordersLoading) {
  return <p>Загрузка...</p>;
}

// error
if (profileError || ordersError) {
  return <p>Ошибка загрузки данных</p>;
}

return (
  <div id="content_profile">

    {/* USER INFO */}
    <div id="info_card">
      <p id="info_card_name">
        Информация о пользователе
      </p>

      <p id="info_card_text">
        {user
          ? `User: ${user.username}`
          : "Нету данных..."}{" "}
```

```
    <br />

    {user
      ? `Имя: ${user.first_name}`
      : "Нету данных..."}{" "}
    <br />

    {user
      ? `Фамилия: ${user.last_name}`
      : "Нету данных..."}
  </p>

  <button
    id="logout_button"
    onClick={handleLogout}
  >
    Выйти из аккаунта
  </button>
</div>

{/* ORDERS */}
<div id="purchase_history_card">
  <p id="purchase_history_card_title">
    История покупок
  </p>

  {orders.length === 0 && (
    <p>Покупок пока нет</p>
  )}

  {orders.map((order, index) => (
    <div
      id="operation_card"
      key={order.id}
    >
      <p id="operation_card_text">
        Покупка №{index + 1}
      </p>

      <p id="operation_card_price">
```

```
            Цена: {order.total_price} ₽
          </p>
        </div>
      ))}
    </div>

  </div>
 );
}

export default ProfilePage;
```

Листинг В.10 — файл RegisterPage.js

```
import React from "react";
import '../styles/forms.css';
import { useForm } from "react-hook-form";
import {Link} from 'react-router-dom';

function RegisterPage() {
  const { register, handleSubmit, formState: { errors } } = useForm();

  async function SubmitForm(data) {
    if (data.password !== data.confirm_password) {
      alert("Пароли не совпадают");
      return;
    }

    const userData = {
      username: data.login_name,
      password: data.password
    };

    console.log("Отправляем на Django:", userData);

    try {
      const response = await fetch("http://localhost:8000/api/register/", {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
```

```
        },
        body: JSON.stringify(userData),
      });

      const result = await response.json();
      console.log("Ответ Django:", result);

      if (response.ok) {
        alert("Регистрация успешна!");
      } else {
        alert("Ошибка: " + (result.detail || "Не удалось создать пользователя"));
      }
    } catch (error) {
      console.error("Ошибка запроса:", error);
      alert("Сервер недоступен");
    }
  }

  return (
    <div id="content_login">
      <form onSubmit={handleSubmit(SubmitForm)}>
        <p id="form_name">Регистрация</p>
        <input
          id="form_input_place"
          type="text"
          placeholder="Логин"
          {...register("login_name", { required: true })}
        />
        {errors.login_name && <p>Введите логин.</p>}

        <input
          id="form_input_place"
          type="password"
          placeholder="Пароль"
          {...register("password", { required: true })}
        />
        {errors.password && <p>Введите пароль.</p>}

        <input
          id="form_input_place"
```

```
            type="password"
            placeholder="Подтвердите пароль"
            {...register("confirm_password", { required: true })}
          />
          {errors.confirm_password && <p>Подтвердите пароль.</p>}

          <input type="submit" value="Зарегистрироваться" id="accept_form_button" />

          <p id="change_form_button"><Link to="/login">Войти</Link></p>
        </form>
      </div>
    );
}

export default RegisterPage;
```

Листинг В.11 — файл ShopPage.js

```
import React, { useEffect } from 'react';
import '../styles/shop.css';

import { useAppDispatch, useAppSelector } from '../app/store/hooks/reduxHooks';
import { fetchGames } from '../app/store/thunks/gamesThunks';
import { fetchCart, addGameToCart } from '../app/store/thunks/cartThunks';

function ShopPage() {

  const dispatch = useAppDispatch();

  const { games, loading, error } = useAppSelector(state => state.games);
  const cartLoading = useAppSelector(state => state.cart.loading);

  useEffect(() => {
    dispatch(fetchGames());
    dispatch(fetchCart());
  }, [dispatch]);

  // добавление в корзину
  const handleBuy = async (id) => {
    const result = await dispatch(addGameToCart(id));
```

```
  if (addGameToCart.fulfilled.match(result)) {
    dispatch(fetchCart());
  }
};

if (loading) return <p>Загрузка...</p>;
if (error) return <p>Ошибка загрузки игр</p>;

return (
  <div id="content_games">

    {games.length === 0 && (
      <div id="game_card">
        <p id="game_card_name">Игр пока нет</p>
        <p id="game_card_text">Зайдите позже</p>
      </div>
    )}

    {games.map(game => (
      <div id="game_card" key={game.id}>
        <p id="game_card_title">{game.title}</p>

        <img
          src={`http://localhost:8000${game.image}`}
          alt="Нет изображения"
          id="game_card_img"
        />

        <p id="game_card_text">{game.description}</p>

        <button
          id="buy_button"
          onClick={() => handleBuy(game.id)}
          disabled={cartLoading}
        >
          Купить: {game.price} ₽
        </button>
      </div>
    ))}
```

```
    </div>
  );
}

export default ShopPage;
```

Листинг В.12 — файл cartApi.js

```
const API_URL = "http://localhost:8000/api";

export const fetchCart = async () => {
  const token = localStorage.getItem("access");
  if (!token) throw new Error("No token");

  const res = await fetch(`${API_URL}/cart/`, {
    headers: { Authorization: `Bearer ${token}` },
  });

  if (!res.ok) throw new Error("Ошибка загрузки корзины");

  return res.json();
};

export const removeCartItem = async (id) => {
  const token = localStorage.getItem("access");

  const res = await fetch(`${API_URL}/cart/remove/`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
    },
    body: JSON.stringify({ item_id: id }),
  });

  if (!res.ok) throw new Error("Ошибка удаления");

  return res.json();
};
```

```
export const buyCart = async () => {
  const token = localStorage.getItem("access");

  const res = await fetch(`${API_URL}/cart/buy/`, {
    method: "POST",
    headers: {
      Authorization: `Bearer ${token}`,
    },
  });

  const data = await res.json();

  if (!res.ok) throw new Error(data.error || "Ошибка покупки");

  return data;
};
```

Листинг В.13 — файл gamesApi.js

```
const API_URL = "http://localhost:8000/api";

export const fetchGames = async () => {
  const res = await fetch(`${API_URL}/games/`);

  if (!res.ok) throw new Error("Ошибка загрузки игр");

  return res.json();
};

export const addGameToCart = async (gameId) => {
  const token = localStorage.getItem("access");

  if (!token) throw new Error("Не авторизован");

  const res = await fetch(`${API_URL}/cart/add/`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
    },
    body: JSON.stringify({ game_id: gameId }),
  });

  if (!res.ok) throw new Error("Ошибка добавления в корзину");

  return res.json();
};
```

Листинг В.14 — файл itemsApi.js

```
const API_URL = "http://localhost:8000/api";

export const fetchItems = async () => {
  const res = await fetch(`${API_URL}/items/`);
```

```
  if (!res.ok) throw new Error("Ошибка загрузки предметов");

  return res.json();
};

export const addItemToCart = async (itemId) => {
  const token = localStorage.getItem("access");

  if (!token) throw new Error("Не авторизован");

  const res = await fetch(`${API_URL}/cart/add/`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
    },
    body: JSON.stringify({ item_id: itemId }),
  });

  if (!res.ok) throw new Error("Ошибка добавления в корзину");

  return res.json();
};
```

Листинг В.15 — файл newsApi.js

```
const API_URL = "http://localhost:8000/api";

export const fetchItems = async () => {
  const res = await fetch(`${API_URL}/items/`);

  if (!res.ok) throw new Error("Ошибка загрузки предметов");

  return res.json();
};

export const addItemToCart = async (itemId) => {
  const token = localStorage.getItem("access");
```

```
  if (!token) throw new Error("Не авторизован");

  const res = await fetch(`${API_URL}/cart/add/`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
    },
    body: JSON.stringify({ item_id: itemId }),
  });

  if (!res.ok) throw new Error("Ошибка добавления в корзину");

  return res.json();
};
```

Листинг В.16 — файл profileApi.js

```
const API_URL = "http://localhost:8000/api";

export const fetchProfile = async () => {
  const token = localStorage.getItem("access");
  if (!token) throw new Error("No token");

  const res = await fetch(`${API_URL}/user/`, {
    headers: { Authorization: `Bearer ${token}` },
  });

  if (!res.ok) throw new Error("Ошибка загрузки профиля");

  return res.json();
};

export const fetchOrders = async () => {
  const token = localStorage.getItem("access");
  if (!token) throw new Error("No token");
```

```
  const res = await fetch(`${API_URL}/orders/`, {
    headers: { Authorization: `Bearer ${token}` },
  });

  if (!res.ok) throw new Error("Ошибка загрузки заказов");

  return res.json();
};
```

Листинг В.17 — файл useCart.js

```
import { useQuery } from "@tanstack/react-query";
import { fetchCart } from "../api/cartApi";

export const useCart = () => {
  return useQuery({
    queryKey: ["cart"],
    queryFn: fetchCart,
    staleTime: 1000 * 60 * 5,
  });
};
```

Листинг В.18 — файл useGames.js

```
import { useQuery } from "@tanstack/react-query";
import { fetchGames } from "../api/gamesApi";

export const useGames = () => {
  return useQuery({
    queryKey: ["games"],
    queryFn: fetchGames,
    staleTime: 1000 * 60 * 5,
  });
};
```

Листинг B.19 — файл useItems.js

```
import { useQuery } from "@tanstack/react-query";
import { fetchItems } from "../api/itemsApi";

export const useItems = () => {
  return useQuery({
    queryKey: ["items"],
    queryFn: fetchItems,
    staleTime: 1000 * 60 * 5,
  });
};
```

Листинг B.20 — файл useLastNews.js

```
import { useQuery } from "@tanstack/react-query";
import { fetchLastNews } from "../api/newsApi";

export const useLastNews = () => {
  return useQuery({
    queryKey: ["lastNews"],
    queryFn: fetchLastNews,
    staleTime: 1000 * 60 * 5,
  });
};
```

Листинг B.21 — файл useNews.js

```
import { useQuery } from "@tanstack/react-query";
import { fetchNews } from "../api/newsApi";

export const useNews = () => {
  return useQuery({
    queryKey: ["news"],
    queryFn: fetchNews,
```

```
    staleTime: 1000 * 60 * 5, // 5 минут
  });
};
```

Листинг B.22 — файл useOrders.js

```
import { useQuery } from "@tanstack/react-query";
import { fetchOrders } from "../api/profileApi";

export const useOrders = () => {
  return useQuery({
    queryKey: ["orders"],
    queryFn: fetchOrders,
  });
};
```

Листинг B.23 — файл useProfile.js

```
import { useQuery } from "@tanstack/react-query";
import { fetchProfile } from "../api/profileApi";

export const useProfile = () => {
  return useQuery({
    queryKey: ["profile"],
    queryFn: fetchProfile,
  });
};
```